# Database

## 모델링부터 인덱스와 캐시까지

무엇을 저장할지 정하고, 빠르게 꺼내는 법 I Software Engineering

**최성철 교수**
**Director of TEAMLAB**

# 방금 프로젝트 이야기를 했습니다

혼자 만들고, 배포하고, 보여 주기

# 어떤 주제든
# 공통인 것이 하나 있습니다

사용자가 넣은 것을 저장했다가, 나중에 다시 보여 준다

# 데이터베이스

# “DB 스키마 짜줘”
# 라고 시키려면

첫 시간 [0] Class Introduction에 있던 슬라이드입니다

# 정규화를 알아야 한다

# SQL 문법은 외우지 않습니다

AI가 씁니다. 대신 AI가 짠 구조가 맞는지는 여러분이 봐야 합니다

# DB 내부도
# 깊이 알 필요는 없습니다

다만 개념은 알아야 합니다. 느려지거나 꼬였을 때 무엇을 물어야 할지 알려면

# 오늘의 차례

1 왜 데이터베이스인가

2 모델링 — 무엇을 저장할까

3 정규화 — 왜 쪼개나

4 DB 안에서 벌어지는 일

5 성능 — 인덱스와 캐시

6 AI가 짠 스키마 검증하기

7 내 프로젝트 ERD

# 오늘의 지도




**설계 → DB 안에서 → 성능. 끝에서 이 그림으로 다시 정리한다.**

# 1. 왜 데이터베이스인가

# 수강신청 시스템을 만든다면

무엇을 저장해야 할까요?

# 학생 · 과목 · 교수 · 학과

# 엑셀 시트 네 장이면 될까?

테이블 = 엑셀 시트. 여기까지는 맞습니다

# 학생이 신청한 과목은 어느 시트에 적을까?

시트와 시트를 잇는 순간 엑셀로는 버거워집니다

# 엑셀 파일과 데이터베이스

| | 엑셀 파일 | 데이터베이스 |
|---|---|---|
| 동시에 | 한 명이 고치면 나머지는 대기 | 수천 명이 동시에 읽고 쓴다 |
| 규칙 | 아무 칸에 아무 값이나 들어간다 | 타입 · 필수 · 관계 규칙을 지킨다 |
| 찾기 | 눈으로 보거나 Ctrl+F | 질의(SQL)로, 수백만 건도 금방 |
| 고장 | 파일이 깨지면 끝 | 기록(로그)으로 되살린다 |

웹 서비스는 여러 사람이 동시에 쓴다. 이 한 줄 때문에 파일이 아니라 DB가 필요하다.

# 데이터베이스는
# 규칙이 있는 공유 저장소

# 관계형 데이터베이스

표(테이블)로 저장하고, 표와 표를 값으로 잇습니다. 1970년 에드거 코드의 아이디어

# 말 몇 개만 정리합니다

| student_id | name | major |
|---|---|---|
| 1 | Kim | SME |
| 2 | Lee | TDE |
| 3 | Park | SME |

표 전체가 테이블, 가로 한 줄(빨강)이 행(레코드), 세로 한 칸이 열(속성). 표의 설계도가 스키마.

# NoSQL은?

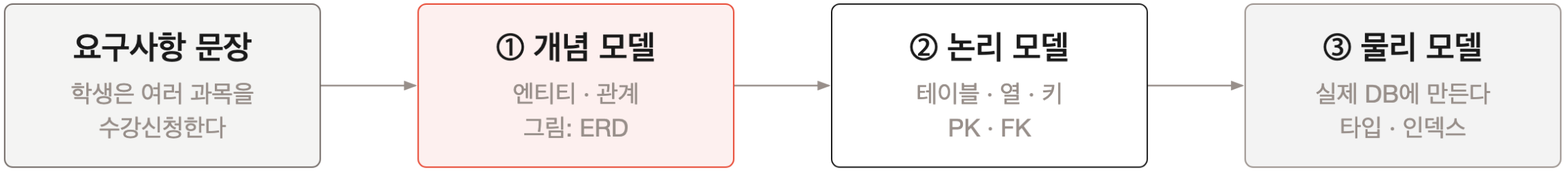
문서 · 키-값 형태로 저장하는 DB도 있습니다. 개념은 같습니다 — 무엇을 어디에 둘지 정해야 합니다

# 2. 모델링

# 코드보다 먼저
# 데이터를 정합니다

화면은 금방 바꿔도, 데이터가 쌓인 테이블 구조는 바꾸기 어렵습니다

# 모델링은 세 단계




오늘 2 · 3장은 ①과 ②, 5장의 인덱스는 ③

**문장 → 그림(ERD) → 표 → 실제 DB. 오른쪽으로 갈수록 바꾸기 어렵다.**

# 요구사항 문장에서 시작합니다

[4] 기획에서 쓴 바로 그 문장들입니다

“학생은 여러 과목을 수강신청한다”

# 문장을 쪼개면

**명사**

학생 · 과목

→ 엔티티 (테이블 후보)

**동사**

수강신청한다

→ 관계

명사는 테이블 후보, 동사는 관계 후보. 가장 쉬운 출발점이다.

# 엔티티

따로 관리할 가치가 있는 대상 — 학생, 과목, 교수, 주문, 게시글

# 속성

엔티티의 성질 — 학생의 이름, 전공, 학년

# “전공”은 엔티티일까, 속성일까?

이름만 적으면 속성. 학과장 · 전화번호 · 위치까지 관리하면 엔티티

키

# 기본 키 (PK)

행 하나를 콕 집는 값. 겹치면 안 되고, 비어 있어도 안 됩니다

# 이름은 기본 키가 될 수 없습니다

동명이인이 있으니까요. 그래서 학번 · 주문번호 같은 번호를 씁니다

# 외래 키 (FK)

다른 테이블의 기본 키를 가리키는 값 — 테이블과 테이블을 잇는 고리

# 관계의 개수

# 관계는 세 가지

| | 뜻 | 예 |
|---|---|---|
| 1 : 1 | 하나에 하나 | 학생 — 학생증 |
| 1 : N | 하나에 여럿 | 학과 — 학생 |
| N : M | 여럿에 여럿 | 학생 — 과목 |

가장 많이 만나는 것은 1:N, 가장 헷갈리는 것은 N:M.

# 1 : N — 학과와 학생

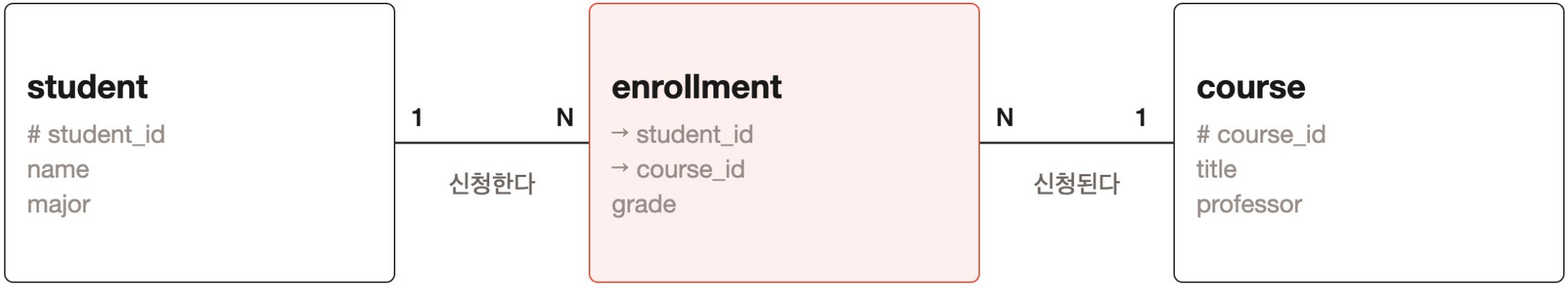



N 쪽(학생)에 외래 키를 둔다. 학과 칸에 학생을 여러 명 적지 않는다.

# N : M은 바로 이을 수 없습니다

학생 칸에 과목을 여러 개, 또는 과목 칸에 학생을 여러 명 적어야 하니까요

# 가운데 표로 푼다




# 기본 키 → 외래 키. 가운데 표의 키는 (student_id, course_id) 두 열을 묶은 것

**N:M 하나를 1:N 두 개로. 학생 1 — 수강신청 N, 과목 1 — 수강신청 N.**

# 가운데 표에도 정보가 생깁니다

성적, 신청 날짜 — 학생의 것도 과목의 것도 아닌, 수강신청의 것

# 직접 만들어 봅시다

# SQLite를 씁니다

파일 하나가 DB 하나. 서버 설치 없이 맥에 기본으로 들어 있습니다

# schema.sql — 세 테이블

```
CREATE TABLE student (
  student_id INTEGER PRIMARY KEY, name TEXT NOT NULL, major TEXT);
CREATE TABLE course (
  course_id TEXT PRIMARY KEY, title TEXT NOT NULL, professor TEXT);
CREATE TABLE enrollment (
  student_id INTEGER REFERENCES student(student_id),
  course_id  TEXT    REFERENCES course(course_id),
  grade TEXT,
  PRIMARY KEY (student_id, course_id));
```

# 만들고, 이어서 꺼내 보면

```
$ sqlite3 school.db < schema.sql
$ sqlite3 school.db ".tables"
course      enrollment  student
$ sqlite3 -header -column school.db "SELECT s.name, c.title, e.grade
    FROM enrollment e JOIN student s ON s.student_id = e.student_id
    JOIN course c ON c.course_id = e.course_id;"
name  title            grade
----  ---------------  -----
Kim   Software Eng.    A
Kim   Database         B
Lee   Software Eng.    A
Park  Operations Res.
```

나눠 저장한 세 표를 JOIN으로 다시 붙여 본다. 저장은 쪼개서, 보여 줄 때는 붙여서.

# 규칙을 어기면 DB가 막는다

```
# 없는 학생(9번)의 수강신청
$ sqlite3 school.db "PRAGMA foreign_keys=ON;
    INSERT INTO enrollment VALUES (9,'SWE','A');"
Error: stepping, FOREIGN KEY constraint failed (19)

# 같은 과목을 두 번 신청
$ sqlite3 school.db "INSERT INTO enrollment VALUES (1,'SWE','C');"
Error: stepping, UNIQUE constraint failed: enrollment.student_id, enrollment.course_id (19)
```

SQLite는 외래 키 검사를 PRAGMA로 켜야 한다. 다른 DB는 보통 처음부터 켜져 있다.

# 규칙은 코드에만 두지 말고
# DB에도 걸어 둡니다

# 자주 거는 규칙

**PRIMARY KEY** — 겹치지 않는 식별자

**FOREIGN KEY** — 없는 대상을 가리키지 못하게

**NOT NULL** — 반드시 채운다

**UNIQUE** — 이메일처럼 겹치면 안 되는 값

**CHECK** — 잔액 ≥ 0 같은 조건

# ERD는 코드로도 그립니다

Mermaid erDiagram — AI가 써 주고, GitHub과 블로그에 그림으로 바로 뜹니다

# Mermaid로 쓴 ERD

```
erDiagram
  STUDENT ||--o{ ENROLLMENT : has
  COURSE  ||--o{ ENROLLMENT : has
  STUDENT { int student_id PK  string name  string major }
  COURSE  { string course_id PK  string title }
  ENROLLMENT { int student_id FK  string course_id FK  string grade }
```

# 3. 정규화

# 처음엔 누구나
# 엑셀 한 장에 다 적습니다

학생 · 과목 · 교수 · 성적을 한 줄에

# 한 장짜리 표 — sheet

```
$ sqlite3 -header -column sheet.db "SELECT * FROM sheet;"
student_id  name   major  course_id  title            professor  grade
----------  ----   -----  ---------  ---------------  ---------  -----
1           Kim    SME    SWE        Software Eng.    Choi       A
1           Kim    SME    DB         Database         Choi       B
2           Lee    TDE    SWE        Software Eng.    Choi       A
3           Park   SME    OR         Operations Res.  Han
```

보기에는 멀쩡하다. 한 줄에 한 번의 수강신청이 들어 있다.

# 뭐가 문제일까요?

같은 사실이 여러 줄에 되풀이됩니다 — Kim의 전공, SWE의 과목명과 교수

# 되풀이되는 칸

| student_id | name | major | course_id | title | professor |
|---|---|---|---|---|---|
| 1 | **Kim** | **SME** | SWE | **Software Eng.** | **Choi** |
| 1 | **Kim** | **SME** | DB | Database | Choi |
| 2 | Lee | TDE | SWE | **Software Eng.** | **Choi** |
| 3 | Park | SME | OR | Operations Res. | Han |

빨간 칸은 다른 줄에 똑같이 또 적혀 있다. 되풀이는 언젠가 어긋난다.

# 이상현상

# ① 수정 이상

SWE 담당 교수가 바뀌었는데, 한 줄만 고쳤다

# 한 줄만 고치면

```
$ sqlite3 sheet.db "UPDATE sheet SET professor='Hong'
    WHERE course_id='SWE' AND student_id=1;"
$ sqlite3 -header -column sheet.db "SELECT DISTINCT course_id, title, professor
    FROM sheet WHERE course_id='SWE';"
course_id  title          professor
---------  -------------  ---------
SWE        Software Eng.  Hong
SWE        Software Eng.  Choi
```

같은 과목인데 교수가 둘이다. 어느 쪽이 맞는지 DB도 모른다.

# ② 삭제 이상

Park이 수강을 취소해서 그 줄을 지웠다

# 학생 한 명을 지웠을 뿐인데

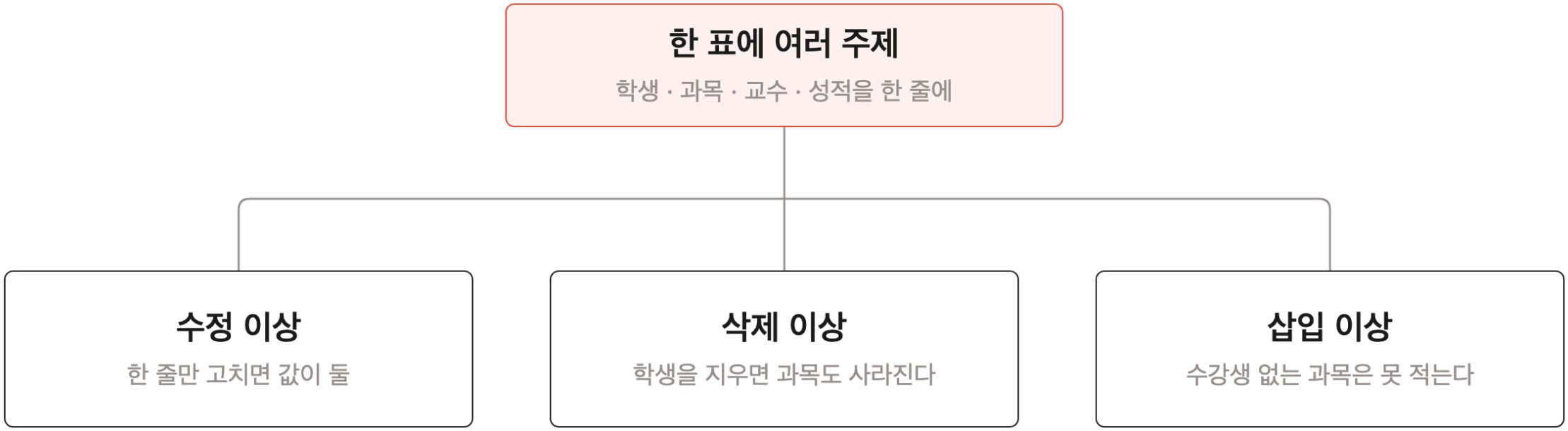
```
$ sqlite3 sheet.db "DELETE FROM sheet WHERE student_id=3;"
$ sqlite3 -header -column sheet.db "SELECT DISTINCT course_id, title FROM sheet;"
course_id  title
---------  -------------
SWE        Software Eng.
DB         Database
```

과목 OR 자체가 사라졌다. 수강생이 없으면 과목도 없는 셈이 된다.

# ③ 삽입 이상

새 과목을 열었는데 아직 신청한 학생이 없다 → 적을 줄이 없다

# 원인은 하나




세 가지 이상현상은 모두 한 표에 여러 주제가 섞인 데서 나온다.

# 정규화는
# 한 표에 한 주제만 남기는 일

# 규칙에 번호가 붙어 있습니다

1NF · 2NF · 3NF. 이름보다 “왜 쪼개나”를 기억하세요

# 1NF — 한 칸에 값 하나

# 한 칸에 여러 개를 적으면

| student_id | name | courses |
|---|---|---|
| 1 | Kim | **SWE, DB** |
| 2 | Lee | SWE |
| 3 | Park | OR |

"SWE를 듣는 학생은?"을 물으려면 글자를 쪼개야 한다. 몇 과목인지 세기도 어렵다.

# 한 줄에 하나씩

| student_id | name | course_id |
|---|---|---|
| 1 | Kim | **SWE** |
| 1 | Kim | **DB** |
| 2 | Lee | SWE |
| 3 | Park | OR |

이게 1NF. 그런데 아직 Kim의 이름이 두 번 적혀 있다 — 다음 규칙으로.

# 2NF — 키 전체에 기댄다

# 이 표의 키는 (student_id, course_id)

둘을 다 알아야 성적이 정해진다. 그런데 과목명은 course_id만 알면 정해진다

# 키의 일부에만 기대는 열

| student_id | course_id | grade | title | professor |
|---|---|---|---|---|
| 1 | SWE | A | **Software Eng.** | **Choi** |
| 1 | DB | B | Database | Choi |
| 2 | SWE | A | **Software Eng.** | **Choi** |

title · professor는 course_id에만 기댄다. 그래서 같은 과목이 나올 때마다 되풀이된다.

# 떼어 냅니다

과목의 것은 과목 표로. 수강신청 표에는 (학번, 과목, 성적)만 남깁니다

# 3NF — 키에만 기댄다

# 키가 아닌 것에 기대는 열

| student_id | name | dept | dept_phone |
|---|---|---|---|
| 1 | Kim | SME | 051-629-0001 |
| 2 | Lee | TDE | 051-629-0002 |
| 3 | Park | SME | 051-629-0001 |

학과 전화번호는 학번이 아니라 학과가 정한다. 학과 번호가 바뀌면 학생 수만큼 고쳐야 한다.

# 역시 떼어 냅니다

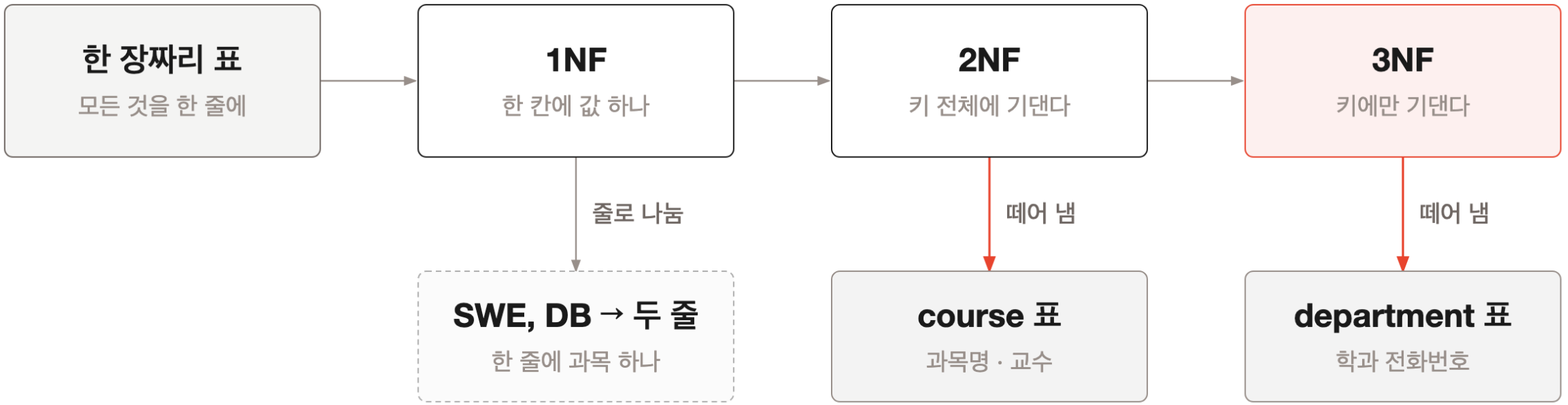
학과의 것은 학과 표로. 학생 표에는 학과를 가리키는 외래 키만

# 세 규칙을 한 장에

| | 규칙 | 어기면 |
|---|---|---|
| 1NF | 한 칸에 값 하나 | 찾고 세기가 안 된다 |
| 2NF | 키 전체에 기댄다 | 과목 정보가 신청마다 되풀이된다 |
| 3NF | 키에만 기댄다 | 학과 정보가 학생마다 되풀이된다 |

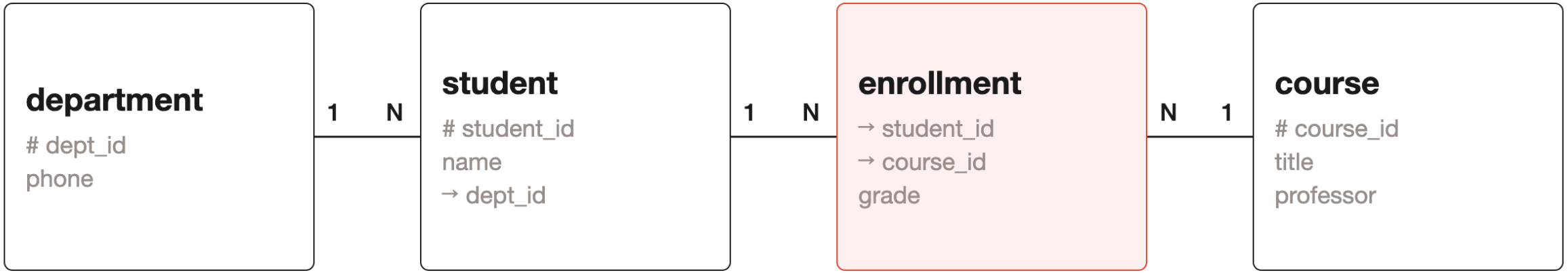
실무에서 “정규화했다”는 말은 보통 3NF까지를 뜻한다.

# 쪼개면 이렇게 떨어져 나온다




단계마다 한 주제씩 떼어 낸다. 남는 것은 각자 자기 주제만 가진 표들.

# 정리하면 네 개의 표




표마다 주제가 하나다. 학과 · 학생 · 수강신청 · 과목.

# 이제 교수 이름은 한 곳만 고친다

```
$ sqlite3 school.db "UPDATE course SET professor='Hong' WHERE course_id='SWE';"
$ sqlite3 -header -column school.db "SELECT s.name, c.title, c.professor
    FROM enrollment e JOIN student s USING(student_id)
    JOIN course c USING(course_id) WHERE c.course_id='SWE';"
name  title          professor
----  -------------  ---------
Kim   Software Eng.  Hong
Lee   Software Eng.  Hong
```

한 번 고쳤는데 두 학생 모두에게 바뀌어 보인다. 어긋날 틈이 없다.

사실 하나는
한 곳에만

# 일부러 다시 합칠 때도 있습니다

반정규화 — 읽기가 아주 많고 느릴 때, 되풀이를 알고서 감수한다

# 순서가 중요합니다

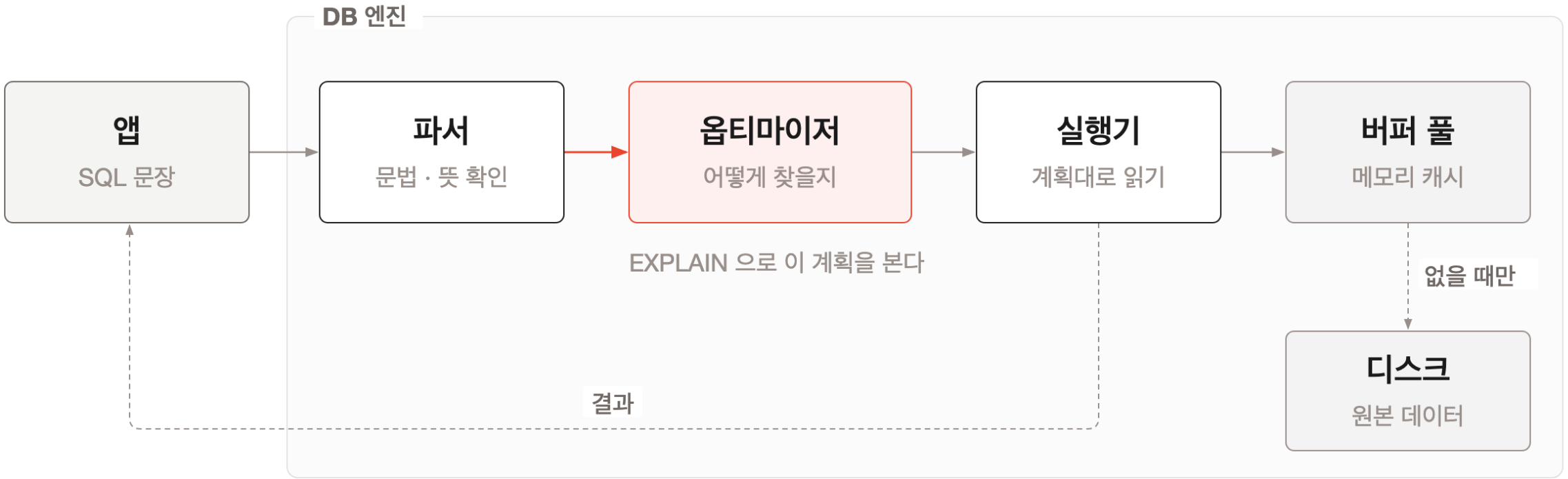
먼저 정규화하고, 느린 것이 확인되면 그때 합칩니다

# 4. DB 안에서 벌어지는 일

# 몰라도 쓸 수는 있습니다
# 그래도 개념은 알아야 합니다

느려지거나 꼬였을 때, AI에게 무엇을 물어야 할지 알려면

# SQL 한 줄이 지나가는 길




우리가 쓰는 것은 맨 왼쪽 한 칸뿐이다. 계획은 옵티마이저가, 캐시는 버퍼 풀이 맡는다.

# SQL은 “무엇을”만 말합니다

“어떻게” 찾을지는 DB의 옵티마이저가 정합니다

# 실행 계획

DB가 고른 “어떻게”. EXPLAIN을 앞에 붙이면 보여 줍니다

# 앞에서 쓴 JOIN의 실행 계획

```
$ sqlite3 school.db "EXPLAIN QUERY PLAN SELECT s.name, c.title
    FROM enrollment e JOIN student s ON s.student_id=e.student_id
    JOIN course c ON c.course_id=e.course_id;"
QUERY PLAN
|--SCAN e USING COVERING INDEX sqlite_autoindex_enrollment_1
|--SEARCH s USING INTEGER PRIMARY KEY (rowid=?)
`--SEARCH c USING INDEX sqlite_autoindex_course_1 (course_id=?)
```

SCAN은 처음부터 끝까지 훑기, SEARCH는 키로 바로 찾기. 이 두 단어만 알아도 된다.

# 데이터는 결국 디스크에 있습니다

그리고 디스크는 메모리보다 훨씬 느립니다

# 얼마나 느린가 (자릿수만 보세요)

| | 걸리는 시간 (대략) | 메모리 읽기를 1초로 치면 |
|---|---|---|
| 메모리 읽기 | 100 나노초 | 1초 |
| SSD 읽기 | 100 마이크로초 | 약 17분 |
| 먼 서버와 한 번 왕복 | 100 밀리초 | 약 12일 |

정확한 값은 장비마다 다르다. 단계마다 천 배씩 느려진다는 감만 가지면 된다.

# DB 성능의 핵심은
# 디스크를 덜 읽는 것

# 버퍼 풀

자주 읽는 데이터 묶음(페이지)을 메모리에 올려 둡니다 — DB 안에 이미 캐시가 있습니다

# 트랜잭션

# 계좌 이체

Lee에게 3만 원 넣고, Kim에게서 3만 원 뺀다. 그런데 Kim 잔액은 1만 원뿐

# 묶지 않고 하면

```
$ cat notx.sql
UPDATE account SET balance = balance + 30000 WHERE name='Lee';
UPDATE account SET balance = balance - 30000 WHERE name='Kim';
$ sqlite3 bank.db < notx.sql
Runtime error near line 2: CHECK constraint failed: balance >= 0 (19)
$ sqlite3 -header -column bank.db "SELECT * FROM account;"
name  balance
----  -------
Kim   10000
Lee   30000
```

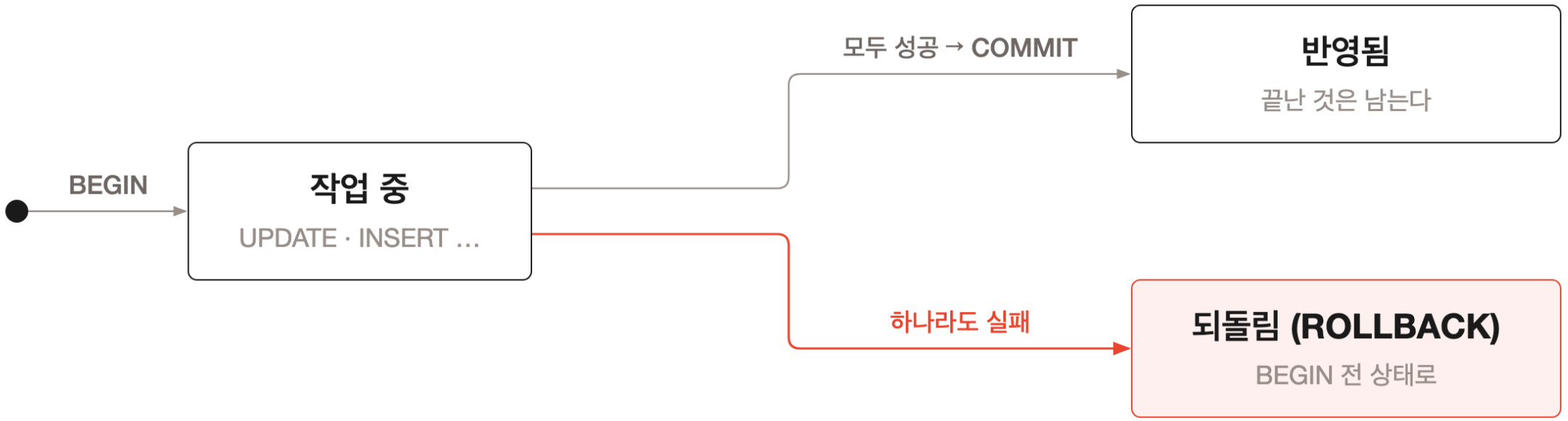
넣기는 성공하고 빼기는 실패했다. 3만 원이 허공에서 생겼다.

# BEGIN으로 묶으면

```
$ cat tx.sql
BEGIN;
UPDATE account SET balance = balance + 30000 WHERE name='Lee';
UPDATE account SET balance = balance - 30000 WHERE name='Kim';
$ sqlite3 -bail bank.db < tx.sql
Runtime error near line 3: CHECK constraint failed: balance >= 0 (19)
$ sqlite3 -header -column bank.db "SELECT * FROM account;"
name  balance
----  -------
Kim   10000
Lee   0
```

뒤가 실패하자 앞의 넣기까지 되돌아갔다. 끝맺지(COMMIT) 못한 트랜잭션은 반영되지 않는다.

# 트랜잭션의 두 갈래




COMMIT 하지 못하고 연결이 끊겨도 되돌아간다

**BEGIN 뒤의 작업은 COMMIT으로 함께 남거나, ROLLBACK으로 함께 사라진다.**

전부 되거나
전혀 안 되거나

# 트랜잭션이 지키는 네 가지 — ACID

| | 뜻 | 예 |
|---|---|---|
| 원자성 | 전부 되거나 전혀 안 되거나 | 이체의 넣기 · 빼기 |
| 일관성 | 규칙(제약)을 늘 지킨다 | 잔액은 0 이상 |
| 격리성 | 동시에 해도 섞이지 않는다 | 두 사람의 동시 이체 |
| 지속성 | 끝난 것은 사라지지 않는다 | 정전 뒤에도 남는다 |

이름은 외우지 않아도 된다. “돈이 허공에서 생기지 않게 해 준다”만 기억하자.

# 두 사람이 동시에 마지막 좌석을 누르면

DB는 잠금(lock)으로 한 사람씩 줄을 세웁니다

# 로그 먼저

고치기 전에 무엇을 고칠지 먼저 기록합니다 — 전원이 나가도 되살리는 이유

# 연결도 비쌉니다

요청마다 새로 접속하면 느리다 → 연결을 미리 몇 개 만들어 돌려 씁니다 (커넥션 풀)

# 여기까지는
# DB가 알아서 해 줍니다

# 우리가 챙길 것은 다음 장

어떤 질의를 보내고, 무엇을 미리 준비해 두느냐

# 5. 성능 — 인덱스와 캐시

# 서비스가 느려지는 원인의 상당수는 DB입니다

그리고 대부분 같은 몇 가지 실수에서 나옵니다

# 인덱스

# 두꺼운 전공 책에서 “트랜잭션”을 찾으려면

첫 쪽부터 넘기거나, 맨 뒤 찾아보기를 펴거나

# 인덱스는
# 책 뒤의 찾아보기

# 훑기와 찾아보기

## 인덱스 없음 — SCAN



한 줄씩 전부 확인한다. 100만 줄이면 100만 번

## 인덱스 있음 — SEARCH




정렬된 나무를 몇 번만 내려가면 닿는다

# B-트리

값을 정렬해 나무 모양으로 묶어 둔 구조. 100만 건이어도 몇 단계만 내려가면 닿습니다

# 직접 재 봅시다

# 주문 100만 건을 만든다

```
$ sqlite3 shop.db "CREATE TABLE orders(order_id INTEGER PRIMARY KEY,
    customer_id INTEGER, amount INTEGER, created_at TEXT); ..."
$ sqlite3 shop.db "SELECT COUNT(*) FROM orders;"
1000000
$ ls -lh shop.db
file size: 25M
# 고객 5만 명이 아무렇게나 주문한 것처럼 만든 가짜 데이터
```

만드는 SQL 전체는 db_demo/demo.sh 에 있다. 출력의 file size 줄은 ls 결과를 줄여 찍은 것.

# 인덱스 없이 고객 4242의 주문을 찾으면

```
$ sqlite3 shop.db < q1.sql      # .timer on
QUERY PLAN
`--SCAN orders
22|1149000
Run Time: real 0.023 user 0.019642 sys 0.003587
```

SCAN — 100만 줄을 전부 봤다. 22건을 찾는 데 0.023초.

# 인덱스를 하나 만들고 다시

```
$ sqlite3 shop.db "CREATE INDEX idx_orders_customer ON orders(customer_id);"
Run Time: real 0.226 user 0.194571 sys 0.027989
$ sqlite3 shop.db < q1.sql
QUERY PLAN
`--SEARCH orders USING INDEX idx_orders_customer (customer_id=?)
22|1149000
Run Time: real 0.000 user 0.000030 sys 0.000046
```

SEARCH — 찾아보기로 바로 갔다. 너무 빨라서 0.000초로 찍힌다.

# 좀 더 정확히 재면 (파이썬으로 200번 평균)

```
$ python3 bench_index.py shop.db
query without index :  22.456 ms / query
query with index    :   0.044 ms / query
insert 100,000 rows without index : 0.028 s
insert 100,000 rows with index    : 0.249 s
```

조회는 빨라졌지만, 입력은 느려졌다. 넣을 때마다 찾아보기도 고쳐야 하기 때문.

# 인덱스의 값

| | 인덱스 없음 | 인덱스 있음 |
|---|---|---|
| 조회 한 번 | 22.456 ms | 0.044 ms (약 500배 빠름) |
| 10만 건 입력 | 0.028 초 | 0.249 초 (약 9배 느림) |
| 파일 크기 | 25 MB | 36 MB |

이 맥에서 SQLite로 잰 값이다. 숫자보다 방향을 보자 — 읽기는 빨라지고, 쓰기와 공간은 비싸진다.

# 인덱스는 공짜가 아닙니다

# 어디에 거나

**WHERE에 자주 나오는 열**

**JOIN에 쓰는 외래 키**

**ORDER BY로 자주 정렬하는 열**

**값이 다양한 열  (두세 가지 값뿐이면 효과가 적다)**

# 기본 키에는 이미 걸려 있습니다

외래 키는 DB마다 다릅니다 — PostgreSQL · SQLite는 자동으로 걸어 주지 않습니다

# 여러 열을 묶은 인덱스는 순서가 있습니다

(성, 이름) 순 전화번호부로 성은 찾지만, 이름만으로는 못 찾습니다

# 열에 함수를 씌우면 못 씁니다

WHERE lower(email) = … 는 email 인덱스를 타지 못합니다

# N+1 문제

# 주문 100개에 고객 이름을 붙이려면

목록 1번 + 고객마다 1번씩 100번 = 101번 묻습니다

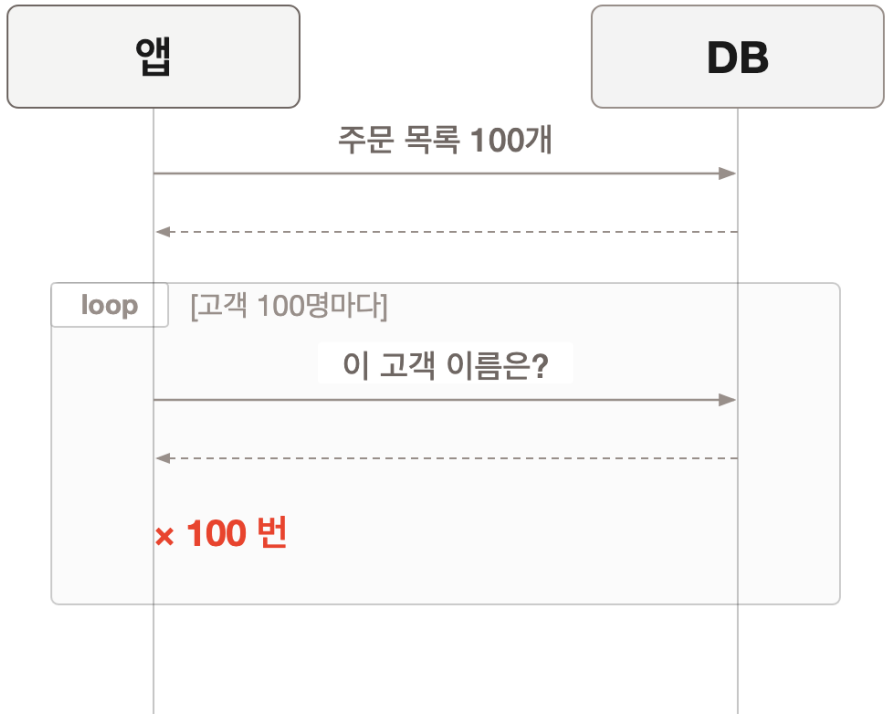
# 두 가지로 짤 수 있다

```
# ① 반복문 — 101번
orders = db.query("SELECT ... FROM orders ORDER BY order_id DESC LIMIT 100")
for o in orders:
    name = db.query("SELECT name FROM customer WHERE customer_id=?", o.customer_id)

# ② JOIN — 1번
db.query("SELECT o.order_id, c.name, o.amount FROM orders o
          JOIN customer c ON c.customer_id = o.customer_id ...")
```

# 묻는 횟수를 그려 보면

**반복문 — 101번**




같은 컴퓨터면 0.46 ms, DB가 멀면 왕복이 101번

**JOIN — 1번**




0.05 ms, 왕복 1번

시간은 아래로 흐른다. 왼쪽은 왕복이 101번, 오른쪽은 1번.

# 결과는 같고, 묻는 횟수가 다르다

```
$ python3 bench_n1_cache.py n1.db n1
loop  :  101 queries,    0.46 ms
JOIN  :    1 query,      0.05 ms
same result: True
```

SQLite는 같은 컴퓨터 안이라 둘 다 빠르다. DB가 다른 서버에 있으면 101번 모두 왕복이 붙는다.

# AI가 짠 코드에서
# 가장 흔한 성능 실수

# 고치는 법

JOIN으로 한 번에, 또는 IN (…)으로 묶어서 두 번에

# 캐시

# 같은 질문을
# 매번 DB에 물어야 할까?

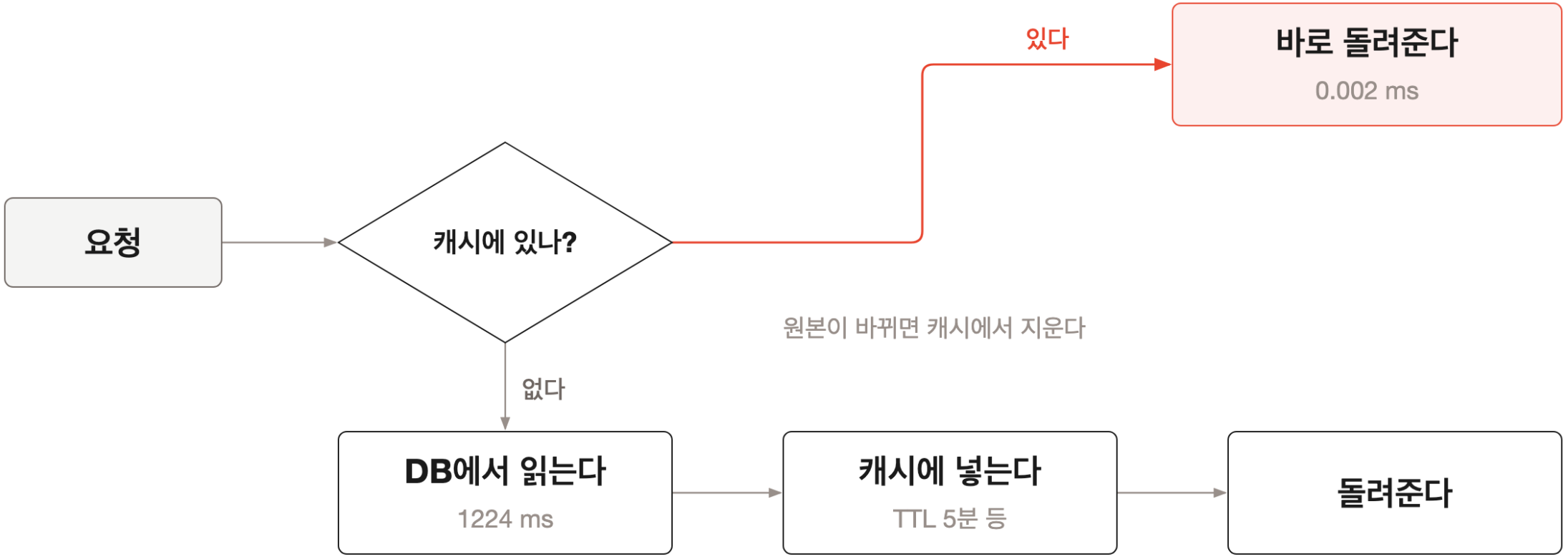
메인 화면의 “이번 주 많이 산 고객 10명”

# 한 번 계산해 두고 꺼내 쓰기

```
$ python3 bench_n1_cache.py n1.db cache
call 1: 1224.640 ms  (DB)
call 2:    0.002 ms  (cache)
call 3:    0.000 ms  (cache)
```

100만 건을 묶어 합계를 내는 데 1.2초. 두 번째부터는 메모리에 둔 결과를 바로 돌려준다.

# 캐시를 쓰는 기본 흐름




먼저 캐시를 보고, 없을 때만 DB에 간다. 원본이 바뀌면 캐시에서 지운다.

# 캐시는 여러 층에 있다




왼쪽에서 답을 찾을수록 빠르다. 디스크까지 내려가는 횟수를 줄이는 것이 목표.

# 캐시의 어려움은
# 언제 버리느냐

# 버리는 두 가지 방법

## 시간으로 (TTL)

5분 지나면 버린다
단순하다

그동안은 옛날 값

## 바뀔 때 (무효화)

원본이 바뀌면 지운다
정확하다

빠뜨리기 쉽다

**둘을 섞어 쓰는 경우가 많다. 바뀔 때 지우되, 놓쳐도 TTL이 지나면 사라지게.**

# 잔액 · 재고는 캐시하지 않습니다

틀리면 안 되는 값은 DB에서 바로. 캐시는 조금 늦어도 괜찮은 값에

# 그 밖에 알아 둘 것

**페이지 나누기** — 1만 건을 한 번에 보내지 않는다

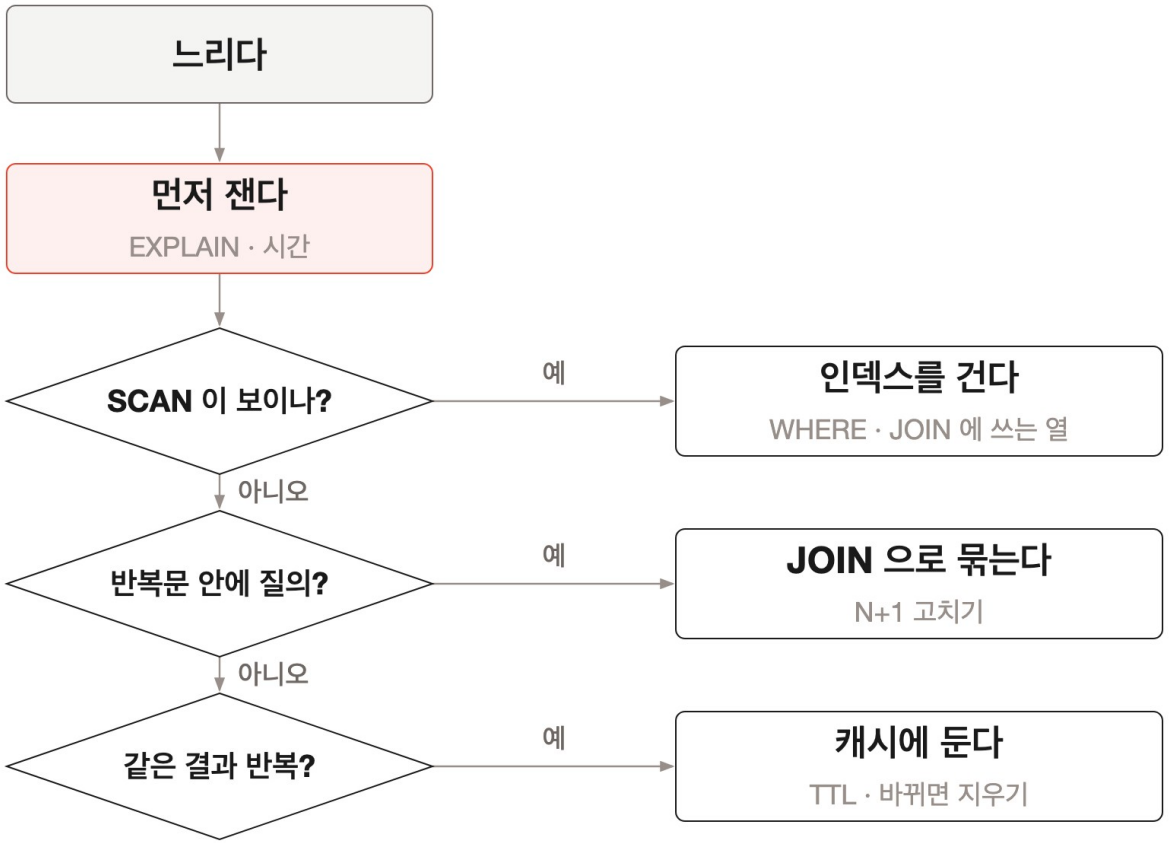
**필요한 열만** — SELECT * 대신 쓸 열을 적는다

**커넥션 풀** — 연결을 만들어 두고 돌려 쓴다

**읽기 복제본** — 읽기가 많으면 복사본에서 읽는다

# 재기 전에
# 고치지 마세요

# 느릴 때 보는 순서




이 수업 프로젝트라면
위 두 개면 충분하다

**짐작으로 고치지 않는다. 재 보고, 원인에 맞는 처방을 고른다.**

# 이 수업 프로젝트 규모라면

외래 키에 인덱스, 목록 화면의 N+1 피하기 — 이 두 가지면 충분합니다

# 6. AI가 짠 스키마 검증하기

# AI는 그럴듯한 스키마를 순식간에 짭니다

그리고 가끔 그럴듯하게 틀립니다

# 이렇게 시킨다

> 동아리 회비를 관리하는 웹 서비스를 만들 거야.
> 회원, 월별 납부 기록, 지출 내역을 저장해야 해.
> 테이블 설계를 3NF로 해 주고, Mermaid ERD로 그려 줘.
> 테이블마다 기본 키, 외래 키, NOT NULL, 인덱스를 적고
> 왜 그렇게 나눴는지 한 줄씩 설명해 줘.

# 이런 표가 나왔다고 해 봅시다

어디가 문제일까요?

# member 테이블

| member_id | name | phone | paid_months | club_name | club_leader |
|---|---|---|---|---|---|
| 1 | Kim | 010-… | 1월,2월,3월 | TEAMLAB | Choi |
| 2 | Lee | 010-… | 1월 | TEAMLAB | Choi |
| 3 | Park | 010-… | | TEAMLAB | Choi |

생각해 보기: 3월 회비를 낸 사람은? 동아리 회장이 바뀌면?

# 문제는 두 군데

| member_id | name | phone | paid_months | club_name | club_leader |
|---|---|---|---|---|---|
| 1 | Kim | 010-… | 1월,2월,3월 | TEAMLAB | Choi |
| 2 | Lee | 010-… | 1월 | TEAMLAB | Choi |
| 3 | Park | 010-… | | TEAMLAB | Choi |

paid_months는 1NF 위반(한 칸에 여러 값) → payment 표로. club_ 두 열은 3NF 위반 → club 표로.

# 검증 체크리스트

1. 모든 테이블에 기본 키가 있다
2. 한 칸에 값 하나 (쉼표로 늘어놓지 않았다)
3. 같은 사실이 두 곳에 적혀 있지 않다
4. N:M은 가운데 표로 풀었다
5. 외래 키 · NOT NULL 같은 규칙을 걸었다
6. 자주 찾는 열과 외래 키에 인덱스가 있다
7. 목록 화면에 N+1이 없다

# 고칠 때도 이유를 묻는다

> 이 스키마에서 같은 사실이 두 곳에 적힌 곳을 찾아서
> 3NF로 고쳐 줘. 바꾼 점마다 어떤 이상현상을 막는지 설명하고,
> 고친 ERD를 Mermaid로 다시 그려 줘.

# 시키는 것은 AI
# 판단은 여러분

# 7. 내 프로젝트 ERD

# 11/9(월)까지 — 블로그 2편

① 내 서비스의 엔티티 3 ~ 5개를 뽑는다

② 관계를 잇고 1:N · N:M을 표시한다

③ Mermaid로 ERD를 그린다

④ 체크리스트로 AI에게 점검받고, 고친 점을 적는다

# 그 전에 11/2(월)까지 — 블로그 1편

주제 제안. 누가, 어떤 불편 때문에, 무엇을, 어디까지

# 오늘 한 것

## 모델링

명사는 엔티티, 동사는 관계

기본 키로 한 줄을 집고, 외래 키로 잇는다

N:M은 가운데 표로 푼다

## 정규화

사실 하나는 한 곳에만

되풀이는 수정 · 삭제 · 삽입 이상을 낳는다

1NF 한 칸 한 값, 2NF 키 전체, 3NF 키에만

## DB 안에서

SQL은 무엇을, 옵티마이저는 어떻게

디스크를 덜 읽는 것이 성능

트랜잭션 — 전부 되거나 전혀 안 되거나

## 성능

인덱스 — 읽기는 빠르게, 쓰기는 느리게

N+1 — 반복문 안의 질의를 의심

캐시 — 언제 버릴지가 어렵다

# 다시, 오늘의 지도


데이터베이스
설계 — 2 · 3장
모델링: 엔티티 · 관계 · 키
정규화: 사실 하나는 한 곳에
DB 안에서 — 4장
실행 계획: SCAN · SEARCH
버퍼 풀: 메모리 캐시
트랜잭션: 전부 아니면 전무
성능 — 5장
인덱스: 읽기 빠르게, 쓰기 느리게
N+1: 반복문 안의 질의
캐시: 언제 버리느냐

# 다음 시간

웹 앱의 구조 — 화면 · 서버 · DB가 어떻게 이어지나

# 감사합니다